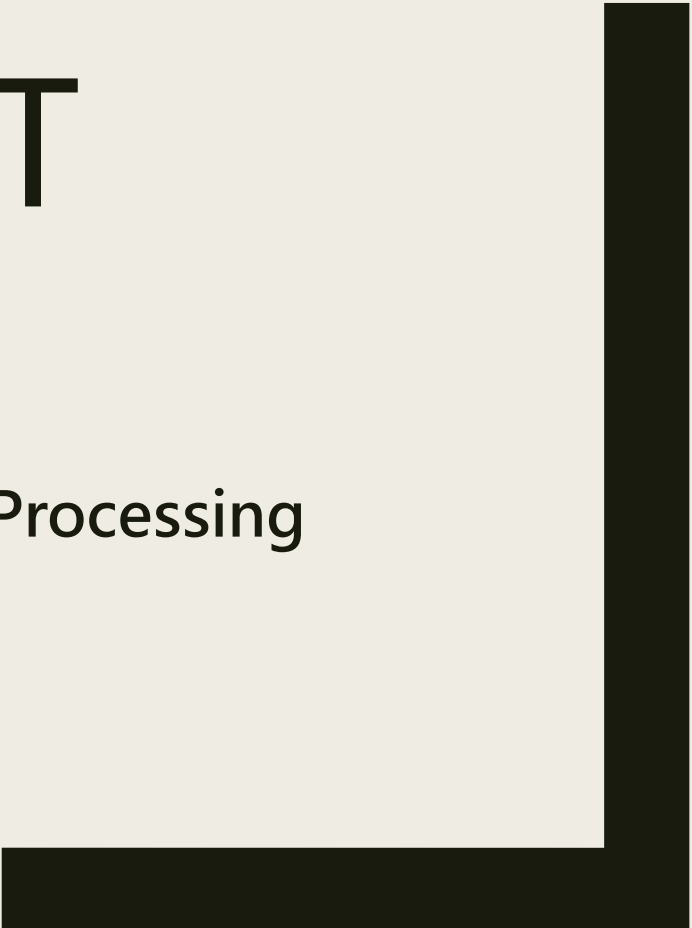




AMR JETBOT TUTORIAL

Lesson 2: JetBot Perception and Image Processing



Course Outline: Week 2

Lesson 2: JetBot Perception and Image Processing

Duration: 2 hours

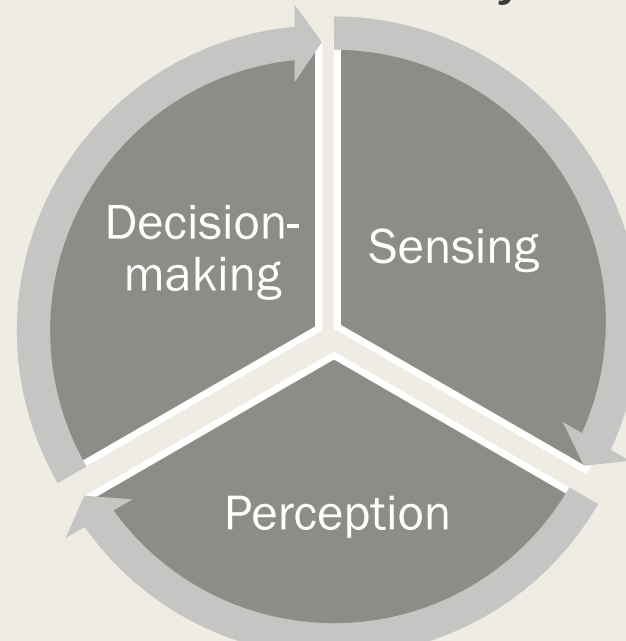
Objective: Understand JetBot's perception system and learn how to use the camera for image processing and object detection.

Outline :

1. **Introduction to JetBot's Camera Function** (15 minute)
 1. Describes how cameras are integrated with JetBot for perception.
 2. Explains the importance of cameras in site navigation (e.g., obstacle detection and path tracking).
2. **OpenCV Basics and Image Processing** (45 minute)
 1. Introduces the basic functions of the OpenCV image processing library and how to use it in Jupyter Notebook.
 2. Use OpenCV to perform basic image reading, display, and simple image processing (such as grayscale conversion and edge detection). °
3. **Object detection and tracking** (45 minute)
 1. Students will learn how to use the JetBot's camera to detect objects and write a tracking program using Python.
 2. Demonstrates how to enable JetBot to perform simple tracking based on objects detected by the camera.
4. **Exercises** (15 minute)
 1. Program JetBot to detect and track objects of a specific color °

Three key elements of a line-following robot

1. Sensing: Acquires raw data from various sensors and analyzes it into meaningful information.
2. Perception: Utilizes existing information to locate the vehicle's current position and analyzes the surrounding environment.
3. Decision-making: Determines the next command based on currently known information and the mission objective.



Install necessary libraries

- `pip install opencv-python`
- `pip install opencv-contrib-python`
- `pip install numpy`
- `pip install Pillow`

First test the camera

- `from jetbot import Camera, bgr8_to_jpeg`
- `import traitlets`
- `import ipywidgets as widgets`
- `from IPython.display import display`
- `import cv2`
- `import numpy as np`
- `from PIL import Image`
- `from io import BytesIO`

- `# Create an Image widget, setting its width and height`
- `image_widget = widgets.Image(format='jpeg', width=500, height=600)`
- `display(image_widget)`

- `camera = Camera.instance()`
- `# Connect the camera's output to the Image widget and convert the format`
- `camera_link = traitlets.dlink((camera, 'value'), (image_widget, 'value'), transform= bgr8_to_jpeg)`
- `print("Camera is now connected to the Image widget.")`
- `camera.stop()`

```
[3]: from jetbot import Camera, bgr8_to_jpeg
import traitlets
import ipywidgets as widgets
from IPython.display import display
import cv2
import numpy as np
from PIL import Image
```

```
[4]: # 創建 Image 小工具，設置寬度和高度
image_widget = widgets.Image(format='jpeg', width=600, height=600)
display(image_widget)

Image(value=b'', format='jpeg', height='600', width='600')
```

```
[5]: camera = Camera.instance()

# 連接攝影機的輸出到 Image 小工具，並轉換格式
camera_link = traitlets.dlink((camera, 'value'), (image_widget, 'value'), transform= bgr8_to_jpeg)

print("攝影機已連接到 Image 小工具。")

攝影機已連接到 Image 小工具。
```

```
[6]: camera.stop()
```

```
[ ]:
```

The camera cannot be turned on normally.

- 1. The camera cable is reversed or not connected properly.
- 2. The program execution order is incorrect.
- 3. Repeatedly starting and declaring the camera (so! camera.stop())
- 4. Bad luck.

Normal screen

```
[10]: import ipywidgets.widgets as widgets
      from jetbot import Robot
      import traitlets

      robot = Robot()

      image = widgets.Image(format='jpeg', width=300, height=300)
      display(image)
```



```
[11]: from jetbot import Camera

      camera = Camera.instance()
```

```
[12]: from jetbot import bgr8_to_jpeg

      camera_link = traitlets.dlink((camera, 'value'), (image, 'value'), transform=bgr8_to_jpeg)
```

camera.value intro

- Numpy's format contains all the data in the photo.
- `frame = camera.value`
- `# Check shape and data type`
- `print("影像形狀:", frame.shape)`
- `print("資料類型:", frame.dtype)`
- `print("BGR 值範圍:", frame.min(), "-", frame.max())`

```
[4]: camera = Camera.instance()

camera_link = traitlets.dlink((camera, 'value'), (image_widget, 'value'), transform= bgr8_to_jpeg)

[6]: frame = camera.value

# 檢查形狀和資料類型
print("影像形狀:", frame.shape)
print("資料類型:", frame.dtype)
print("BGR 值範圍:", frame.min(), "-", frame.max())
```

https://www.waveshare.com/wiki/IMX219-160_Camera

Solving the issue of the purple frame on the camera

1. Download the camera-override.isp file and extract it to a specific folder:

```
wget https://files.waveshare.com/upload/e/eb/Camera_overrides.tar.gz
tar zxvf Camera_overrides.tar.gz
sudo cp camera_overrides.isp /var/nvidia/nvcam/settings/
```

2. Install files:

```
sudo chmod 664 /var/nvidia/nvcam/settings/camera_overrides.isp
sudo chown root:root /var/nvidia/nvcam/settings/camera_overrides.isp
```

```
wget https://files.waveshare.com/upload/e/eb/Camera_overrides.tar.gz
tar zxvf Camera_overrides.tar.gz
sudo cp camera_overrides.isp /var/nvidia/nvcam/settings/
```

```
sudo chmod 664 /var/nvidia/nvcam/settings/camera_overrides.isp
sudo chown root:root /var/nvidia/nvcam/settings/camera_overrides.isp
```

You cannot directly
modify it using jupyter;
you must modify it
directly in the Jason
Nano console!!!



Using OpenCV to process images: Grayscale demonstration

- `gray_widget = widgets.Image(format='jpeg', width=600, height=600)`
- `# 顯示小工具`
- `display(widgets.HBox([image_widget, gray_widget]))`
- `def update_images(change):`
- `frame = change['new']`
- `rgb_frame = frame`
- `_, img_encoded = cv2.imencode('.jpg', rgb_frame)`
- `image_widget.value = img_encoded.tobytes()`
- `gray_frame = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)`
- `_, gray_encoded = cv2.imencode('.jpg', gray_frame)`
- `gray_widget.value = gray_encoded.tobytes()`
- `camera.observe(update_images, names='value')`

```
# 創建 Image 小工具, 設置寬度和高度
image_widget = widgets.Image(format='jpeg', width=600, height=600)
gray_widget = widgets.Image(format='jpeg', width=600, height=600)

display(widgets.HBox([image_widget, gray_widget]))
```

```
def update_images(change):

    frame = change['new']

    rgb_frame = frame
    _, img_encoded = cv2.imencode('.jpg', rgb_frame)
    image_widget.value = img_encoded.tobytes()

    gray_frame = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
    _, gray_encoded = cv2.imencode('.jpg', gray_frame)
    gray_widget.value = gray_encoded.tobytes()
```

```
camera.observe(update_images, names='value')
```

Edge detection using the Canny algorithm

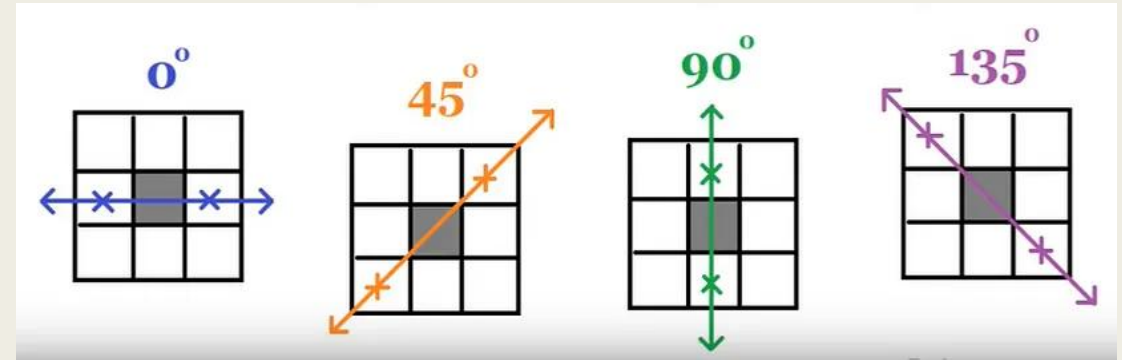
- `edge_widget = widgets.Image(format='jpeg', width=600, height=600)`
- `display(widgets.HBox([image_widget, gray_widget, edge_widget]))`

- `threshold1_slider = widgets.IntSlider(min=0, max=500, value=100, description='Threshold1')`
- `threshold2_slider = widgets.IntSlider(min=0, max=500, value=200, description='Threshold2')`
- `display(widgets.HBox([threshold1_slider, threshold2_slider]))`

- `edges = cv2.Canny(gray_frame, threshold1_slider.value, threshold2_slider.value)`

- `# Encode the edge image into JPEG format and update the display.`
- `_, edge_encoded = cv2.imencode('.jpg', edges)`
- `edge_widget.value = edge_encoded.tobytes()`

Canny algorithm



- The functions of Threshold1 and Threshold2 :
 - **Threshold1 (low threshold) :**
 - Sets the threshold value for weak edges.
 - Pixels with gradient strength below Threshold1 are considered not edges and are discarded.
 - **Threshold2 (high threshold) :**
 - Sets the threshold value for strong edges.
 - Pixels with gradient strength above Threshold2 are considered strong edges and are retained.

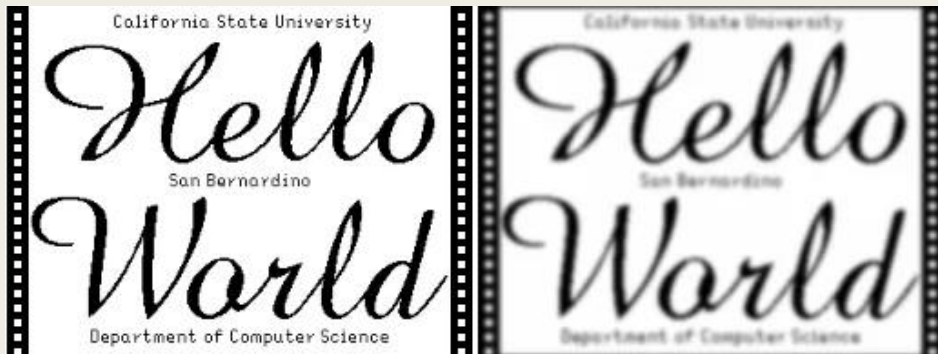
$$\begin{bmatrix} 0.5 & 0.9 & 1 \\ 0 & 0.3 & 0.7 \\ 0.9 & 0 & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} 0.5 & 0.9 & 1 \\ 0 & \text{red } 0.3 & 0.7 \\ \text{red } 0.9 & 0 & 0 \end{bmatrix}$$

Gaussian filter

In the field of image processing, there are many different types of filters, which can be broadly divided into the following two categories:

- a. Smoothing (noise removal): Mean filter, Median filter, Gaussian filter
- b. Sharpening (edge enhancement): Laplace filter, Sobel filter

The uses of Gaussian: **reducing image noise** and **lowering the level of detail**, its visual effect is like viewing an image through a semi-transparent screen.



```
■ gray_frame = cv2.GaussianBlur(gray_frame, (5, 5), 0)
```

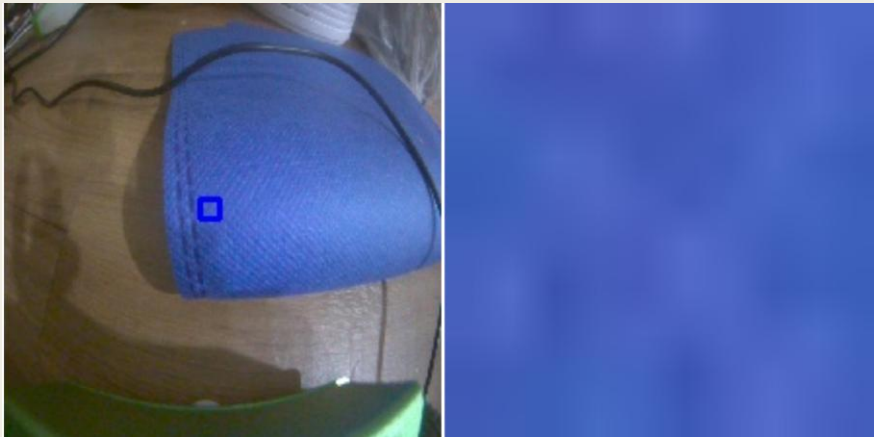
Gaussian Filter Matrix

The following is an approximate 3x3 Gaussian filter generalized weighted smoothing filter matrix. Convolution of the image with a 3x3 Gaussian filter will achieve the effects of filtering out noise, low-pass filtering, and blurring.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Drawing ROI

- `roi_widget = widgets.Image(format='jpeg', width=600, height=600)`
- `# Set the position and size of the ROI`
- `roi_x, roi_y, roi_w, roi_h = 60, 40, 15, 15`
- `mean_label = widgets.Label()`
- `# Display the widget`
- `display(widgets.HBox([image_widget, roi_widget]), mean_label)`



```
def update_images(change):  
    # Get a new image frame (assuming RGB format)  
  
    frame = change['new']  
  
    # Convert the image from RGB to BGR format for OpenCV use  
  
    bgr_frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)  
  
    # Draw a ROI rectangle on the image (red box; in BGR format, red is (0, 0, 255))  
  
    cv2.rectangle(bgr_frame, (roi_x, roi_y), (roi_x + roi_w, roi_y + roi_h), (0, 0, 255), 2)  
  
    # Convert the image back from BGR to RGB format for display  
  
    rgb_frame_with_rect = cv2.cvtColor(bgr_frame, cv2.COLOR_BGR2RGB)  
  
    # Convert the entire image to JPEG Format and update the display  
  
    _, img_encoded = cv2.imencode('.jpg', rgb_frame_with_rect)  
  
    image_widget.value = img_encoded.tobytes()  
  
    # Extract the ROI region (directly extracted from the original RGB image)  
  
    roi_frame = frame[roi_y:roi_y + roi_h, roi_x:roi_x + roi_w]  
  
    # Calculate the average RGB value of the ROI  
  
    mean_rgb = roi_frame.mean(axis=(0, 1))  
  
    b_mean = mean_rgb[2]  
  
    g_mean = mean_rgb[1]  
  
    r_mean = mean_rgb[0]  
  
    # Update the value of mean_label  
  
    mean_label.value = f"ROI average RGB value: R={r_mean:.2f}, G={g_mean:.2f}, B={b_mean:.2f}"  
  
    # Convert ROI image to JPEG format and update display  
  
    _, roi_encoded = cv2.imencode('.jpg', roi_frame)  
  
    roi_widget.value = roi_encoded.tobytes()
```

Binarization

Used for color recognition, it sets an RGB threshold for a specific color, and filters out colors based on the threshold. The final image will resemble a black and white photo, where black represents all colors and white represents the color of interest.

```
r_lower = widgets.IntSlider(min=0, max=255, value=0, description='R Lower')
r_upper = widgets.IntSlider(min=0, max=255, value=255, description='R Upper')
g_lower = widgets.IntSlider(min=0, max=255, value=0, description='G Lower')
g_upper = widgets.IntSlider(min=0, max=255, value=255, description='G Upper')
b_lower = widgets.IntSlider(min=0, max=255, value=0, description='B Lower')
b_upper = widgets.IntSlider(min=0, max=255, value=255, description='B Upper')
```

```
display(widgets.VBox([r_lower, r_upper, g_lower, g_upper, b_lower, b_upper]))
```



```
# Get the current RGB range values of the slider
```

```
lower_rgb = np.array([b_lower.value, g_lower.value, r_lower.value])
```

```
upper_rgb = np.array([b_upper.value, g_upper.value, r_upper.value])
```

```
# Use the inRange function to binarize and filter pixels within the specified RGB
```

```
mask = cv2.inRange(frame, lower_rgb, upper_rgb)
```

```
# Convert the binarized mask image to RGB format for easier display
```

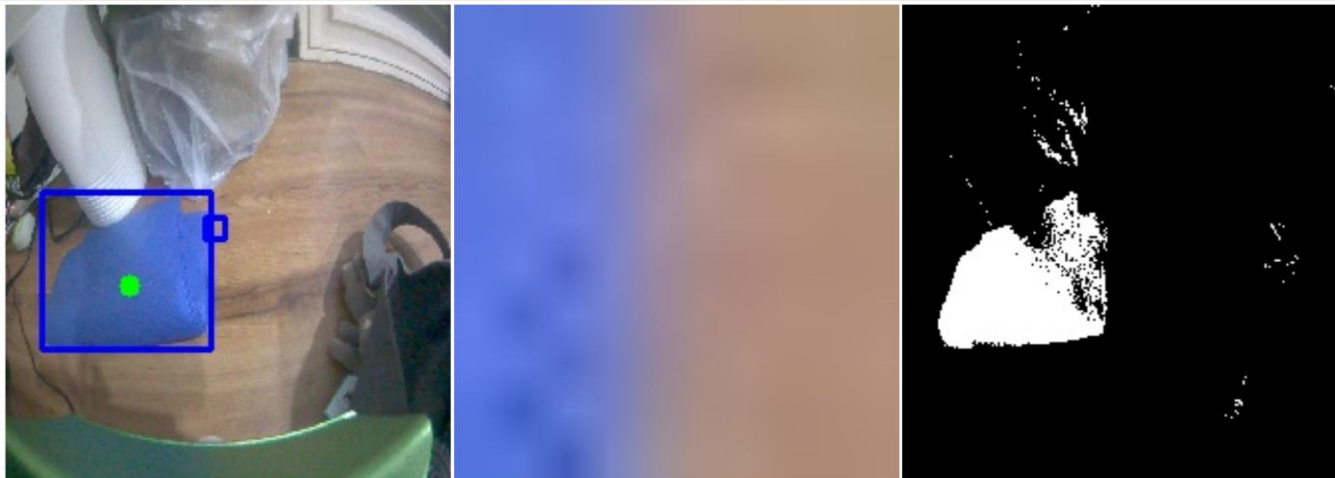
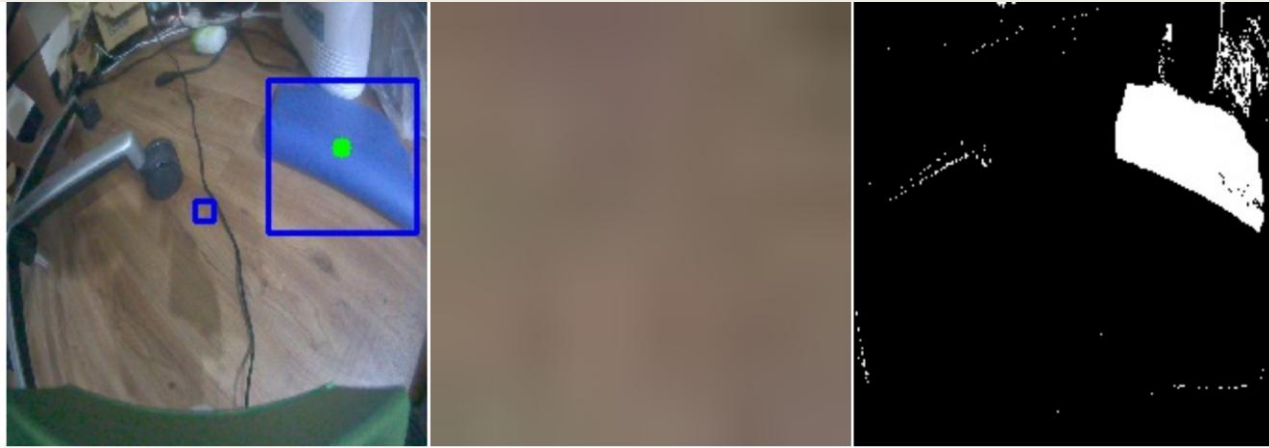
```
mask_rgb = cv2.cvtColor(mask, cv2.COLOR_GRAY2RGB)
```

```
_, mask_encoded = cv2.imencode('.jpg', mask_rgb)
```

```
binary_widget.value = mask_encoded.tobytes()
```

Binarization for object recognition

By using findContours to detect the contours of binarized values, and then filtering them, objects of a specific color can be found through binarization.



Binarized object recognition code

```
# # Create sliders for minimum and maximum area thresholds
```

```
min_area_slider = widgets.IntSlider(value=500, min=0, max=10000, step=50, description=' Minimum area ')
```

```
max_area_slider = widgets.IntSlider(value=5000, min=0, max=20000, step=50, description=' Maximum area ')
```

```
display(widgets.VBox([r_lower, r_upper, g_lower, g_upper, b_lower, b_upper, min_area_slider, max_area_slider]))
```

```
contours, _ = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
# Get minimum and maximum area thresholds from sliders
```

```
min_area = min_area_slider.value
```

```
max_area = max_area_slider.value
```

```
# Initialize lists to store center points and bounding boxes
```

```
centroids = []
```

```
bounding_boxes = []
```

```
for cnt in contours:
```

```
    area = cv2.contourArea(cnt)
```

```
    if min_area <= area <= max_area:
```

```
        # Calculate center points
```

```
        M = cv2.moments(cnt)
```

```
        if M['m00'] != 0:
```

```
            cx = int(M['m10']/M['m00'])
```

```
            cy = int(M['m01']/M['m00'])
```

```
            centroid = (cx, cy)
```

```
        # Get bounding box
```

```
        x, y, w, h = cv2.boundingRect(cnt)
```

```
        bbox = (x, y, w, h)
```

```
        # Check if it overlaps with an existing bounding box
```

```
        overlap = False
```

```
        for existing_bbox in bounding_boxes:
```

```
            x1, y1, w1, h1 = existing_bbox
```

```
            if (x < x1 + w1 and x + w > x1 and y < y1 + h1 and y + h > y1):
```

```
                overlap = True
```

```
                break
```

```
            if not overlap:
```

```
                centroids.append(centroid)
```

```
                bounding_boxes.append(bbox)
```

```
        # Draw the center point on the original image
```

```
        cv2.circle(rgb_frame_with_rect, centroid, 5, (0, 255, 0), -1)
```

```
        # Optional: Draw the bounding box
```

```
        cv2.rectangle(rgb_frame_with_rect, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

```
        # Encode and update the image with the center point drawn
```

```
        _, img_encoded = cv2.imencode('.jpg', rgb_frame_with_rect)
```

```
        image_widget.value = img_encoded.tobytes()
```


第二周結束 -

